Federico Saporiti, MSc in Control Systems Engineering
January 15, 2026

1 Objective of the Program

The objective of this program is to detect road lanes and roadway area/lane area from an input image using classical computer vision techniques. It will also identify whether the lane line is solid or dashed and, finally, these components will be integrated to enable the program to operate on Video 2.
Finally, it will try to detect cars on frame 100 of video 1.

2 Line Detection - step by step Algorithm

First, the original image is processed using the Canny blur pipeline. This process consists of three main steps, each of which serves a specific purpose in preparing the image for lane detection:

- Firstly, the image is converted from BGR (standard for cv2) to grayscale. This reduces the image to a single intensity channel, eliminating color information that is not essential for edge detection. Working in grey scale simplifies the computation and highlights intensity variations, which are crucial for identifying edges.
- Next, a Gaussian blur with a kernel size of 5x5 is applied to the grayscale image. Gaussian blurring is a smoothing operation that reduces noise and intensity fluctuations by convolving the image with a Gaussian filter. The chosen kernel size balances noise reduction and edge preservation, ensuring that lane boundaries are detectable while irrelevant details are suppressed.
- Canny detects edges by first calculating the variation in pixel intensity between neighboring pixels to estimate image gradients. These gradients indicate areas where the intensity changes sharply, which typically correspond to edges. The algorithm then suppresses non-maximum gradient values to produce thin, well-defined edges and applies a double-threshold mechanism to distinguish strong edges from weak ones. Weak edges are retained only if they are connected to strong edges, resulting in a clear and reliable edge map suitable for further processing.

Before applying the Hough transform, the image is cropped to a region of interest. This region is defined by a triangular area that roughly corresponds to where lanes are expected to appear on the road. This removes irrelevant portions of the image, such as the sky or surrounding scenery, reducing noise and focusing line detection on the road only. We will demonstrate on the first image how applying different regions of interest affects the lane detection results. Changing the shape or position of the triangular ROI causes the program to select different subsets of the image for analysis, potentially leading to the identification of different line segments.

The *HoughLinesP* algorithm is then applied to the edge-detected image to identify line segments corresponding to lane markings. This method is based on the probabilistic Hough transform, which detects line segments by analysing the geometric alignment of edge points. Rather than examining all possible points, the algorithm randomly samples a subset of edge pixels and maps them into a parameter space defined by line parameters (ρ, θ). In this space, each edge point

votes for all lines that could pass through it. Lines are identified as peaks in the accumulator, corresponding to sets of edge points that are approximately collinear. The probabilistic approach reduces computational complexity, and this will be very important when we will run the algorithm on the entire video.
The parameters of function *cv.HoughLinesP* are:

- ρ: Distance resolution of the accumulator in pixels. It determines the precision with which the distance of a line from the origin is measured.
- θ: Angular resolution of the accumulator in radians. It defines the smallest angle step used to detect line orientations.
- *threshold*: Minimum number of votes required in the accumulator for a line to be considered valid.
- *minLineLength*: Minimum length that a line segment must have to be accepted. This parameter helps discard short segments that are likely caused by noise.
- *maxLineGap*: Maximum distance allowed between two line segments for them to be connected into a single line. This enables the detection of continuous lane markings even when edges are partially discontinuous.

After the line segments have been detected, an averaging procedure is applied to obtain a single, representative line for each lane. First, the detected lines are classified as left or right based on the sign of their slope: negative slopes correspond to the left lane, while positive slopes correspond to the right lane. For each group, the slope and intercept of the line segments are computed and averaged to reduce noise and variability among detections.

The averaged slope and intercept values are then converted into pixel coordinates to produce one consolidated line for each lane. Finally, these lane lines are overlaid on a copy of the original image using a weighted sum between the original image and the blank image with averaged lines drawn only.

Lane area is then shown by filling the polygon created by the averaged lanes, using the *cv.fillPoly* function and doing a weighted sum with the original frame as before.

Figure 1: road1

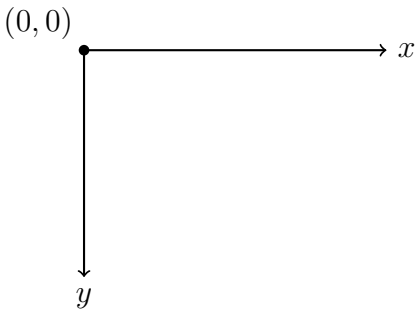
3 Line Type Detection

Idea:
In order to detect whether a line is solid or dashed, we will proceed as follows: We will draw a fixed number of equally spaced horizontal lines (50 lines demonstrated to be sufficient in our case) and observe how many intersections we get per number of lines with the **raw** Hough lines (the averaged ones are always continuous). We will call this ratio *hit_ratio*.

Algorithm:
First of all, we need to distinguish raw Hough lines between left and right lines. This is done by computing the sign of the line slope.

The standard pyplot axes are defined as follows:

- The origin $(0, 0)$ is at the **top-left**
- The y-axis increases **downward**
- The x-axis increases to the **right**



So, it comes naturally that positive sloped lines will be classified as right lines and negative ones as left lines. I called the function that computes all this *split_lane_by_side*.

Then we build the function *checkLine_yc*, which generates the equally spaced horizontal lines on the lower part of the image. I choose to generate it from the bottom up to 60% of the image height because not only it is a generally sufficient portion to understand wether the line is dashed or solid, but also because the Hough algorithm becomes generally imprecise in detecting lines far from the origin of the image, due to decreasing definition.

The lane marking type is determined by the value *hit_ratio*. The function *classify_line_type* performs this task by testing a set of evenly spaced horizontal check lines against the segments. For each check line, *checkLine_intersects* evaluates all raw Hough line segments and produces a Boolean array indicating whether the check line intersects at least one segment per side. An intersection is considered to occur whenever the check line falls between the vertical endpoints of a segment, allowing for a small tolerance to account for noise or small gaps in detection.
The resulting Boolean array thus encodes, for every check line, whether a segment is present at that y coordinate. Solid lane markings intersect most check lines, producing a largely true Boolean array, while dashed lanes naturally create repeated gaps where no segment intersects the check lines, resulting in alternating true and false entries. From this array, two metrics are computed: the fraction of check lines intersected by at least one segment (*hit_ratio*) and the total number of gaps. Finally, if the hit ratio exceeds a threshold of 0.7, the lane is classified as solid; otherwise, it is classified as dashed. This method has demonstrated to be able to effectively highlight the structural difference between solid and dashed lines in our examples.

I applied this algorithm to image *road1*, then to frames of *video1*, and then to frame 200 of *video2*. Then I performed a real time analysis on *video2*. I had to adjust the region of interest for the frames of *video1* and *video2*, as well as in the video to maximize the accuracy of all algorithms involved.

For *video2* frame 200, I also had to make the *avg_lines* algorithm more robust by performing some checks before calculations were performed (everything is explained in the commented code). For this video, I also had to change the minimum hit ratio at which we determine whether a line is dashed or not. The difference in hit ratio between a solid and dashed line is still significant, and I have included this in the video.

Figure 2: road1

Figure 3: frame 400 of video1

4 Car Detection Using Harris Corner Detection

The proposed algorithm detects cars in road scenes by leveraging the structural characteristics of vehicles, particularly the presence of strong corners.

Within the ROI, the image is converted to grayscale and transformed into a floating-point representation to satisfy the requirements of the Harris Corner Detection algorithm. Harris corner detection is then applied to identify points with significant intensity variation in multiple directions, which commonly occur at vehicle edges, windows, and structural boundaries. A threshold is applied to the corner response to generate a binary mask highlighting strong corner locations.

Since individual corner points are insufficient to represent complete vehicles, morphological closing is performed on the binary mask to spatially group neighboring corners into compact regions. This operation effectively merges clusters of corner responses into larger blobs that are more representative of vehicle-sized objects.

Contours are extracted from these grouped regions, and each contour is evaluated using simple geometric constraints. Specifically, the contour area and aspect ratio of its bounding box are used to filter out noise, road markings, and other non-vehicle structures. Only contours whose dimensions are consistent with typical car shapes are retained as valid detections.

Finally, the bounding boxes corresponding to valid detections are projected back onto the original image by accounting for the ROI offsets. The detected vehicles are visualized by drawing bounding rectangles and labels on the full-resolution frame.

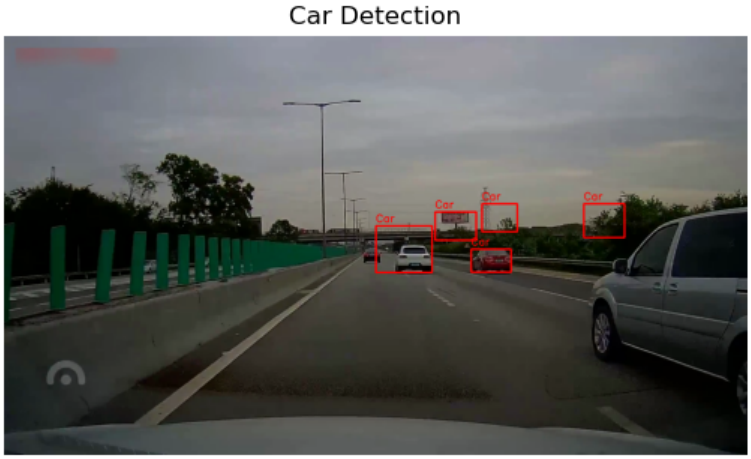


Figure 4: frame 100 of video1

5 Cons and Possible Developments of the Algorithms Used

The algorithms adopted in this work offer a simple and computationally efficient solution for lane and vehicle-related feature detection. However, they also present several limitations that suggest possible directions for future improvements. One of the main drawbacks is the need to manually define the Region of Interest (ROI). A possible development would be the automatic or adaptive identification of the ROI based on scene understanding, camera calibration, or vanishing point estimation.

Another limitation arises from the use of the Hough Transform for line detection, which is inherently sensitive to noise and image disturbances. Irregular textures, shadows, or road artifacts can generate spurious edge responses that lead to false line detections. Although geometric constraints and thresholding partially mitigate this issue, they do not completely eliminate false positives in complex scenes.

In the video-based implementation, the temporal nature of the data offers both challenges and opportunities. The current approach relies on a hit ratio computed over multiple frames to validate detected lines. However, due to the chosen ROI starting from approximately 60% of the image height, the hit ratio threshold had to be lowered. This is because the actual road region occupies only 30–40% of the vertical image dimension. Despite the fact that I've not included this adjustment, the algorithm performs well thanks to the fact that the ROI effectively excludes many sources of disturbance, allowing to effectively reach a huge difference in hit_ratio between solid and dashed lines (and of course instead of a threshold, I could just do a comparison between values of hit_ratio of both lines to detect whether the line is solid or dashed, because in some video frames, the hit ratio is around 0.4, while the other is 0 and it detects the first line as dashed).

Another possible improvement in the video for line type detection could be the following: instead of detecting multiple lines and validating them solely based on spatial hit ratios, a single control line could be used and evaluated across a fixed number of frames over a sufficiently long temporal window (e.g., 20/30 frames). Continuous lane markings would intersect the control line consistently, whereas dashed marking detections would result in intermittent intersections. This frame-based hit ratio would provide a more robust discrimination criterion and reduce sensitivity to noise, at the cost of losing precision for the first few frames of the video.